

2026

# DOCUMENTACIÓN TÉCNICA

PROYECTO PINGU

Riddler Company



## Contents

|                                               |    |
|-----------------------------------------------|----|
| 1. Introduction .....                         | 3  |
| 2. Overall architecture .....                 | 4  |
| 3. API Endpoints .....                        | 6  |
| 3.1. List of available endpoints .....        | 7  |
| 3.2. Examples of requests and responses ..... | 8  |
| 4. Database .....                             | 15 |
| 4.1. Entity-Relationship Diagram.....         | 15 |
| 4.2. Creating the database.....               | 15 |
| 4.3. Description of tables and fields.....    | 16 |
| 4.4. Important Rules.....                     | 17 |
| 4.5. Migration Management.....                | 17 |
| 5. Web application .....                      | 18 |
| 5.1. Technology used .....                    | 18 |
| 5.2. Project Structure.....                   | 18 |
| 5.3. Connecting to the API .....              | 19 |
| 5.4. Data validation .....                    | 20 |
| 6. Mobile App .....                           | 21 |
| 6.1. Technology used .....                    | 21 |
| 6.2. Project structure .....                  | 21 |
| 6.3. Connecting to the API .....              | 22 |
| 6.4. Session Management.....                  | 22 |
| 6.5. Validations .....                        | 22 |
| 7. Desktop application .....                  | 24 |
| 7.1. Technology used .....                    | 24 |
| 7.2. Project structure .....                  | 24 |
| 7.3. Connecting to the API .....              | 25 |
| 7.4. Session Management.....                  | 25 |
| 7.5. Validations .....                        | 25 |
| 8. Security .....                             | 27 |
| 8.1. Security mechanisms.....                 | 27 |
| 8.2. User and permissions management.....     | 27 |
| 8.3. Best practices.....                      | 27 |
| 9. Deployment.....                            | 28 |

|                                          |    |
|------------------------------------------|----|
| 9.1. How to run each application.....    | 28 |
| 9.2. Deploying the API .....             | 28 |
| 9.3. Environments .....                  | 28 |
| 10. Testing .....                        | 29 |
| 10.1. Testing of the API.....            | 29 |
| 10.2. Client-side testing .....          | 29 |
| 10.3. Error testing and validation ..... | 29 |
| 11. User and Maintenance Guide .....     | 30 |
| 11.1. Local execution .....              | 30 |
| 11.2. Adding new features .....          | 30 |
| 11.3. Common issues.....                 | 30 |
| 11.4. Maintenance.....                   | 31 |
| 12. Future improvements .....            | 32 |

# 1. Introduction

Today, social media apps are part of the daily lives of millions of people and provide a comprehensive example for putting software development knowledge into practice. These types of systems allow for the integrated development of interface design, inter-application communication, database access, security, data validation and the organisation of business logic. For this reason, they are particularly well-suited for a Final Year Project.

This project has developed PingU, a small multi-platform social network that allows users to register, log in, post content, reply to posts, like or unlike posts, follow other users, view followers and those they follow, edit their profile and recover their password. The main feature of the system is that all these functionalities are supported by a single client-server architecture, comprising a REST API and a shared database, to which three different clients connect: a web application, a mobile app for Android and a desktop application in C#.

The main objective of the project has been to design and implement a functional and consistent system across different platforms, reusing the same data access logic and ensuring similar behaviour across all clients. In this way, the work is not limited to the development of a single interface, but takes a broader view of software development, where multiple applications consume the same central service.

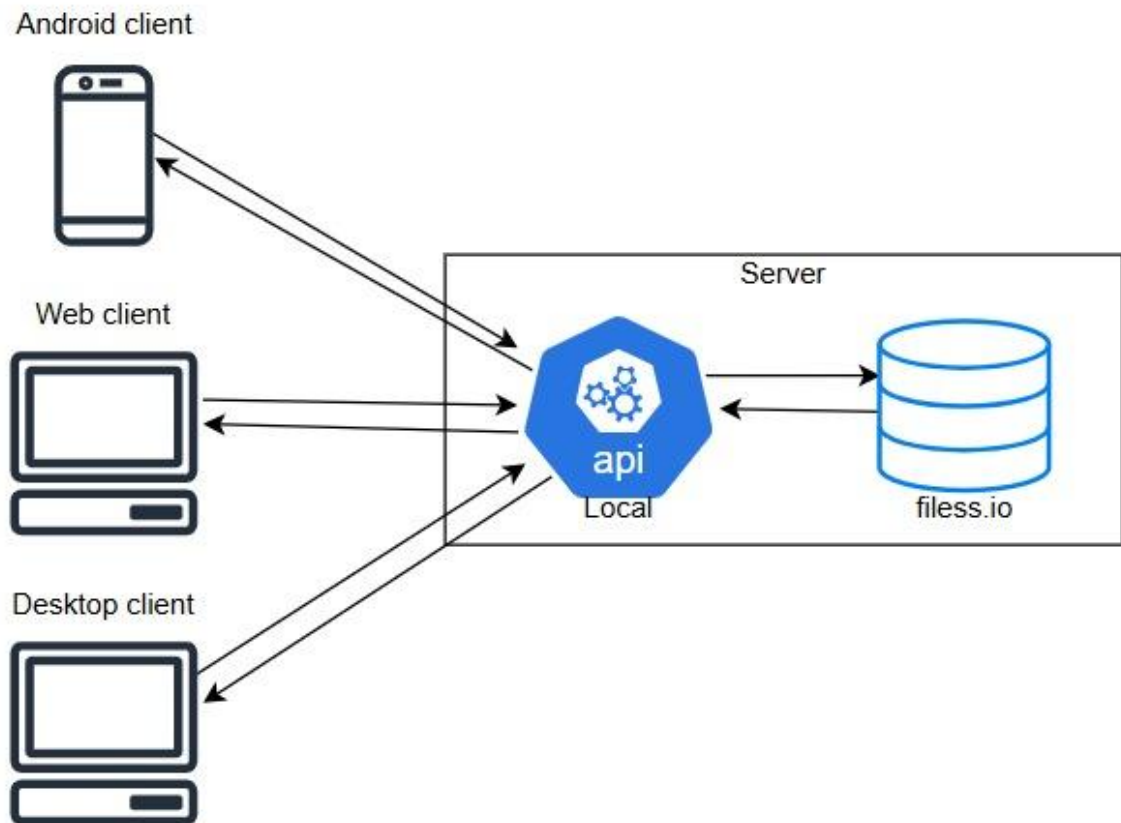
To this end, a REST API in Java has been created and deployed on Apache Tomcat, a relational database responsible for storing users, posts, follow relationships and reactions, and three clients adapted to their respective runtime environments. The web version has been developed using HTML, CSS and pure JavaScript; the mobile app using Java in Android Studio; and the desktop app using C# and Windows Forms. This combination has enabled the application of different technologies within the same functional domain, whilst also comparing different approaches to building interfaces and managing user sessions.

Throughout this document, we describe the system's overall architecture, the API endpoints, the database design, the implementation of the three clients, the security measures adopted, the on-premises deployment, the tests carried out, and potential areas for future improvement. Taken as a whole, this work reflects a comprehensive, functional and scalable project, aimed not only at the development of a specific application, but also at providing a comprehensive understanding of how a service-based, multi-platform system is built and organised.

## 2. Overall architecture

The system architecture proposed for this final-year project is based on a client-server model, in which three client applications (web, mobile and desktop) utilize a shared server, which contains a REST API for accessing the business logic and a database where all the information is stored.

### Client-server architecture used in PingU



Firstly, the system features a database hosted in a network-accessible environment, responsible for storing all information relating to users, posts, social connections (followers/followed) and reactions. This database acts as a centralized persistence layer for all clients.

Built on top of this database is the REST API, developed in Java and deployed on an Apache Tomcat server running locally (port 8080). This API forms the core of the system, as it exposes the various endpoints that enable the management of users, authentication, posts, interactions and relationships between users. In this way, the business logic is completely decoupled from the three user interfaces.

The various clients developed consume this API in a uniform manner, using the following base URL:

<http://localhost:8080/api/rest/pingu/>

In the case of the Android app, the address 10.0.2.2 is used instead of localhost, as the emulator redirects this IP address to the host machine's localhost.

The system comprises three distinct clients:

- A web application developed using HTML, CSS and JavaScript.
- A mobile app for Android.
- A desktop application developed in C# (Windows Forms).

Each of these clients implements its own user interface, adapted to the environment in which it runs, but they all share the same data access logic via the REST API.

As for session management, this is handled locally on each client, without using a centralized authentication system with persistent tokens. Specifically:

- In the web application, the session is stored in the browser's localStorage.
- In the Android application, SharedPreferences is used.
- In the desktop application, the session is maintained via static variables in memory whilst the application is running.

Finally, this architecture offers several clear advantages. Firstly, it allows all information to be decoupled from the business logic of the different clients, making it easier to maintain control over the information and carry out maintenance on each part separately. Furthermore, it ensures that the three clients operate consistently using the same system and the same information. Finally, security is improved as the data is not distributed across each device.

### 3. API Endpoints

The PingU application relies on a single REST API, which centralizes the system's logic and acts as a communication hub between the database and the various client applications. All data exchanges are carried out in JSON format, both in requests and responses.

The base URL used by the API is as follows:

`http://localhost:8080/api/rest/pingu/`

In the case of the Android application, the address 10.0.2.2 is used instead of localhost, as the emulator interprets that IP address as the host computer's localhost.

The API implements operations related to authentication, user management, tracking relationships and content publishing. Different HTTP methods are used for this, depending on the action to be performed. Thus, GET is used to retrieve information, POST to create resources or trigger specific actions not included in the REST standard, PUT to update existing data and DELETE to remove resources.

As for authentication, the system uses a login based on a username and password. When the user enters their credentials, the client sends a request to the API, which validates the information received and returns the user's identifier if access is granted. From that point onwards, each client maintains the session locally in accordance with its own platform's mechanism. Furthermore, the API includes a specific password recovery endpoint, which allows a new password to be generated and sent to the email address associated with the account.

### 3.1. List of available endpoints

All API endpoints use JSON. The main endpoints implemented and used by the system's various clients are listed below.

#### Authentication

| Method      | Route               | Description                            |
|-------------|---------------------|----------------------------------------|
| <b>POST</b> | /auth/login         | Log in with your username and password |
| <b>POST</b> | /auth/pass-remember | Reset password via email               |

#### Users

| Method        | Route                 | Description              |
|---------------|-----------------------|--------------------------|
| <b>POST</b>   | /users                | Register a new user      |
| <b>GET</b>    | /users                | Get a list of all users  |
| <b>GET</b>    | /users/{id}           | Get user data            |
| <b>PUT</b>    | /users/{id}           | Update user data         |
| <b>DELETE</b> | /users/{id}           | Delete account           |
| <b>GET</b>    | /users/{id}/followers | Get the user's followers |
| <b>GET</b>    | /users/{id}/followed  | Get followers            |
| <b>POST</b>   | /users/{id}/follow    | Follow a user            |
| <b>POST</b>   | /users/{id}/unfollow  | Unfollow a user          |



## Publications

| Method | Route               | Description           |
|--------|---------------------|-----------------------|
| GET    | /posts              | View all posts        |
| POST   | /posts              | Create a new post     |
| DELETE | /posts/{id}         | Delete a post         |
| GET    | /posts/{id}/replies | Get replies to a post |
| GET    | /posts/{id}/likes   | Get likes on a post   |
| POST   | /posts/{id}/like    | Like a post           |
| POST   | /posts/{id}/dislike | Unlike a post         |

## 3.2. Examples of requests and responses

Below are some examples illustrating how the API works.

### Login

Request

```
POST /auth/login
Content-Type: application/json
{
  "alias": "atlas",
  "contrasena": "VDoYRNteGx"
}
```

Response

```
{
  "message": "Login correcto",
  "id": "2"
}
```

This endpoint validates the user's credentials and, if they are correct, returns their identifier, which will be used by clients to keep the session active.

## Password recovery

### Request

```
POST /auth/pass-remember
Content-Type: application/json
{
  "correo_electronico": "carlositaliani@gmail.com"
}
```

### Response

```
{
  "message": "Se ha enviado una nueva contraseña al email"
}
```

This endpoint allows you to initiate the password recovery process if you have forgotten your password.

## User registration

### Request

```
POST /users
Content-Type: application/json
{
  "alias": "oterixXx3",
  "nombre_visible": "juan otero otero",
  "correo_electronico": "prueba369@prueba.com",
  "contrasena": "123456"
}
```

### Response

```
{
  "message": "Usuario creado exitosamente",
  "id": "28"
}
```

This endpoint creates a new account in the system and returns the user ID generated for the user.

## Get the list of users

### Request

```
GET /users
Accept: application/json
```

### Response

```
[
  {
    "alias": "Atlas",
    "nombre_visible": "Carlos Alberto Piñeiro Italiani",
    "correo_electronico": "Carlositaliani@gmail.com",
    "biografia": "estudiante de segundo año DAM",
  }
]
```

```
"fotografia": "",  
"user_id": "7",  
"fecha_alta": null  
}  
]
```

This endpoint returns the list of registered users, along with the information needed to build screens such as the general user list.

### Retrieve a user's followers

Request

```
GET /users/6/followers  
Accept: application/json
```

Response

```
[ ]
```

In this case, the user in question has no followers at this time.

### Get followers

Request

```
GET /users/6/followed  
Accept: application/json
```

Response

```
[  
  {  
    "idSeguidor": 6,  
    "idSeguido": 1,  
    "fecha": 1773183600000  
  },  
  {  
    "idSeguidor": 6,  
    "idSeguido": 3,  
    "fecha": 1773183600000  
  },  
  {  
    "idSeguidor": 6,  
    "idSeguido": 4,  
    "fecha": 1773183600000  
  }  
]
```

This endpoint allows you to retrieve the tracking relationships for a specific user.

### Follow a user

Request

```
POST /users/3/follow  
Content-Type: application/json
```

```
{
  "id_seguidor": 3
}
```

Response

```
{
  "message": "Desde ahora sigues a este usuario"
}
```

This call creates a new tracking relationship between two users.

### Unfollow a user

Request

```
POST /users/3/unfollow
Content-Type: application/json
{
  "id_seguidor": 3
}
```

Response

```
{
  "message": "Dejaste de seguir al usuario"
}
```

This endpoint deletes an existing tracking relationship.

### Get posts

Request

```
GET /posts
Accept: application/json
```

Response

```
[
  {
    "id": "5",
    "id_autor": "1",
    "contenido": "Este es el primer post",
    "urlMultimedia": null,
    "idPostPadre": null,
    "fecha_creacion": 1771110000000
  }
]
```

This endpoint returns the system's main posts, i.e. the root posts visible on the home screen.

### Create a post

Request

```
POST /posts
Content-Type: application/json
{
  "contenido": "segunda respuesta al post padre",
  "id_autor": "1",
  "idPostPadre": "5",
  "urlMultimedia": null
}
```

#### Response

```
{
  "message": "Post creado",
  "id": "7"
}
```

This endpoint is used both to create a new post and to post a reply to another post, depending on whether or not the parent post's ID is provided.

#### Get replies to a post

##### Request

```
GET /posts/5/replies
Accept: application/json
```

##### Response

```
[
  {
    "id": "6",
    "id_autor": "1",
    "contenido": "primera respuesta al post padre",
    "urlMultimedia": null,
    "idPostPadre": "5",
    "fecha_creacion": 1771110000000
  },
  {
    "id": "7",
    "id_autor": "1",
    "contenido": "segunda respuesta al post padre",
    "urlMultimedia": null,
    "idPostPadre": "5",
    "fecha_creacion": 1771110000000
  }
]
```

This call retrieves the responses associated with a specific post.

#### Get likes on a post

##### Request

```
GET /posts/6/likes
Accept: application/json
```

##### Response

```
[
  {
    "idUsuario": "1",
    "id_post": "6",
    "fecha_creacion": 1771110000000
  },
  {
    "idUsuario": "2",
    "id_post": "6",
    "fecha_creacion": 1771110000000
  }
]
```

This endpoint returns the list of positive reactions to a post and allows you to calculate the number of likes.

### Like a post

Request

```
POST /posts/6/like
Content-Type: application/json
{
  "idUsuario": 2
}
```

Response

```
{
  "message": "Like registrado"
}
```

### Unlike a post

Request

```
POST /posts/5/dislike
Content-Type: application/json
{
  "idUsuario": 1
}
```

Response

```
{
  "message": "Unlike realizado"
}
```

### Delete a post

Request

```
DELETE /posts/1
Accept: application/json
```

Response

```
{  
  "message": "Post eliminado correctamente"  
}
```

This endpoint allows you to delete an existing post from the system.

### **Delete a user account**

Request

```
DELETE /users/1  
Accept: application/json
```

Response

```
{  
  "mensaje": "Usuario eliminado correctamente"  
}
```

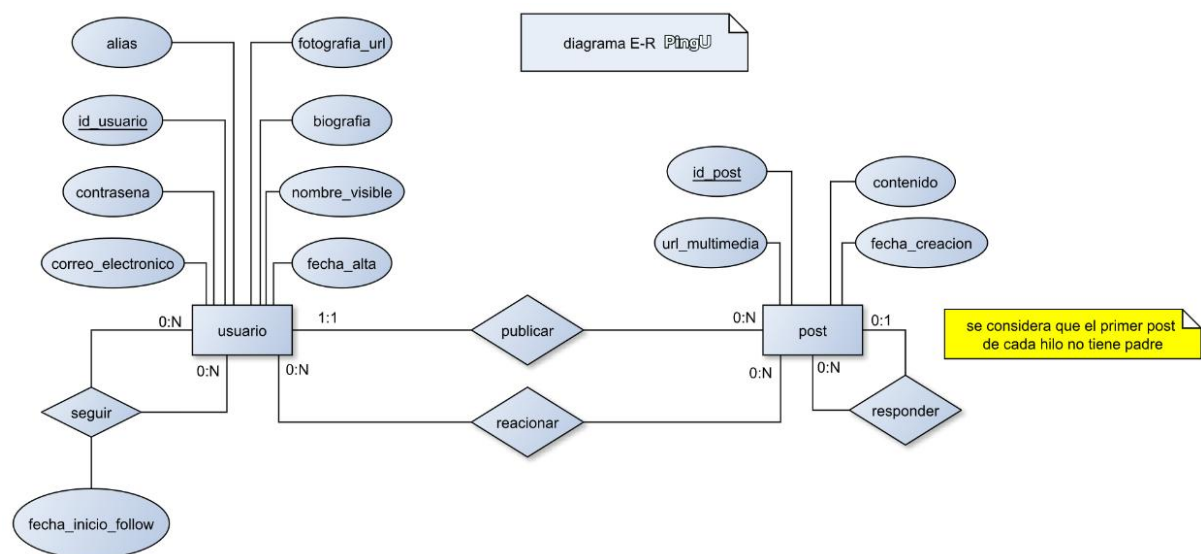
This operation removes a user's account from the system.

## 4. Database

The PingU database is responsible for storing all the information required for the application to function. It stores user data, posts, follow relationships between accounts, and reactions to posts.

### 4.1. Entity-Relationship Diagram

The database's entity-relationship model reflects the system's main entities and the relationships between them. The central tables are USER and POST, around which the rest of the relationships are structured. On the one hand, the FOLLOW table represents the relationship between users who follow other users. On the other hand, the REACT table records the likes given by users on posts. In this way, the model allows for the representation of both the platform's basic information and the social interactions between its participants.



### 4.2. Creating the database

#### -- 1. USER table

```
CREATE TABLE USER (  
  user_id INT AUTO_INCREMENT PRIMARY KEY,  
  alias VARCHAR(50) NOT NULL,  
  display_name VARCHAR(100),  
  registration_date DATE,  
  email_address VARCHAR(100) UNIQUE NOT NULL,  
  biography TEXT,  
  password VARCHAR(255) NOT NULL,  
  photo_url VARCHAR(255) -- optional  
);
```

#### -- 2. POST table

```
CREATE TABLE POST (  
  post_id INT AUTO_INCREMENT PRIMARY KEY,
```



```

content TEXT NOT NULL,
media_url VARCHAR(255), -- optional
creation_date DATE,
author_id INT NOT NULL,
parent_post_id INT, -- optional (may be NULL if not a reply)
-- Definition of foreign keys
CONSTRAINT fk_post_author FOREIGN KEY (author_id) REFERENCES USER(user_id)
ON DELETE CASCADE,
CONSTRAINT fk_post_parent FOREIGN KEY (parent_post_id) REFERENCES
POST(post_id) ON DELETE SET NULL
);

```

### -- 3. FOLLOW table

```

CREATE TABLE FOLLOW (
follower_id INT NOT NULL,
followed_id INT NOT NULL,
start_date_follow DATE,
PRIMARY KEY (follower_id, followed_id),
CONSTRAINT fk_follow_follower FOREIGN KEY (follower_id) REFERENCES
USER(user_id) ON DELETE CASCADE,
CONSTRAINT fk_follow_followed FOREIGN KEY (followed_id) REFERENCES
USER(user_id) ON DELETE CASCADE
);

```

### -- 4. REACTION table

```

CREATE TABLE REACTION (
user_id INT NOT NULL,
post_id INT NOT NULL,
reaction_date DATE,
PRIMARY KEY (user_id, post_id),
CONSTRAINT fk_reaction_user FOREIGN KEY (user_id) REFERENCES USER(user_id)
ON DELETE CASCADE,
CONSTRAINT fk_reaction_post FOREIGN KEY (post_id) REFERENCES POST(post_id)
ON DELETE CASCADE
);

```

## 4.3. Description of tables and fields

The USUARIO table stores the basic information for each account registered in the system. Its primary key is *id\_usuario*, an auto-incrementing numeric identifier. It also includes the alias field, which visibly identifies the user within the application; *nombre\_visible*, used as the name displayed in the interface; *fecha\_alta*, which records the date the account was created; and *correo\_electronico*, which must be unique to avoid duplicates. It also stores the *biografia*, the encrypted password and, optionally, the *fotografia\_url*.

The POST table stores posts made by users. Its primary key is *id\_post*. Each post contains mandatory *contenido*, a possible *url\_multimedia*, the *fecha\_creacion* and the author's identifier in the *id\_autor* field, which acts as a foreign key to the USER table. It also includes the *id\_post\_padre* field, which allows replies to other posts to be represented. If this field is null, it means that this is a main post; if it has a value, it indicates that the post is a reply.

The SEGUIR table represents the follow relationship between two users. It consists of the *id\_seguidor* and *id\_seguido* fields, which together form the composite primary key. This prevents the same relationship from being duplicated. It also includes *fecha\_inicio\_follow*, which records when the follow began. Both fields are foreign keys pointing to the USER table.

The REACCIONAR table records the likes a user gives to a post. Its primary key is also composite, consisting of *id\_usuario* and *id\_post*, which prevents the same user from liking the same post more than once. It also includes the *fecha\_reaccion* field, which stores the date on which the interaction took place. Its two main fields are foreign keys to the USER and POST tables, respectively.

#### 4.4. Important Rules

Several important rules have been established in the database design to ensure data consistency. Firstly, each user's email address must be unique, which prevents two accounts from being registered with the same email address. Similarly, the FOLLOW and REACT tables use composite primary keys to prevent duplicates in follow relationships and likes.

Cascading deletion rules have also been defined. If a user is deleted, their posts, follow relationships and reactions are automatically deleted, as all this data depends on their existence. In the case of posts, if a post with associated replies is deleted, the *id\_post\_padre* field of those replies is set to *NULL*, thus preventing the child messages from being lost entirely, even though they are no longer linked to their original post.

Another relevant rule is that all posts must have a valid author, so *id\_autor* cannot be null. Similarly, every reaction must be associated with an existing user and an existing post. This ensures that no orphaned or inconsistent relationships can be stored within the system.

#### 4.5. Migration Management

No automated migration tools were used during the course of the project. The database schema was created and modified using SQL statements executed manually on the database.

In this project, the database structure was managed in a controlled manner using table creation scripts and ad hoc modifications as and when required during development. This approach proved sufficient as it was a small-scale project with a limited number of structural changes. However, in a real-world environment or a larger-scale project, it would be advisable to incorporate a versioned migration system that would allow for maintaining a change history, automating deployments and reducing errors in the evolution of the schema.

## 5. Web application

The PingU web application is one of the system's three clients and allows users to access all the main features of the social network via a web browser. It has been designed to be simple, without the use of frameworks, with the aim of maintaining full control over the logic and making it easier to understand how it works internally.

### 5.1. Technology used

The web application has been developed using HTML, CSS and pure JavaScript, without relying on frameworks or external libraries. This approach allows for a detailed understanding of how each part of the application is built and how communication with the API takes place.

Each screen consists of three separate files:

- An .html file that defines the structure
- A .css file that handles the visual design
- A .js file where all the logic is implemented

This separation facilitates project organization and improves code maintainability.

### 5.2. Project Structure

The application is organized into different screens, each corresponding to a specific system function. Each screen has its own HTML, CSS and JavaScript (with the exception of Riddler.html, which has no functionality, and recuperarContraseña.html, which contains its functionality within the HTML), ensuring that each view is completely independent.

| File           | Description                                                                  |
|----------------|------------------------------------------------------------------------------|
| index.html     | Login screen with a login form and options to save your username or password |
| registro.html  | Registration of new users with client-side validation                        |
| home.html      | Main feed showing all posts, including like and reply options                |
| perfil.html    | Viewing and editing the logged-in user's data                                |
| misPost.html   | List of your own posts with the option to delete them                        |
| nuevoPost.html | Form for creating new posts                                                  |

|                          |                              |
|--------------------------|------------------------------|
| usuarios.html            | List of all registered users |
| followers.html           | List of the user's followers |
| follows.html             | List of the user's follows   |
| recuperarContrasena.html | Password recovery form       |
| Riddler.html             | Developer's website          |

### 5.3. Connecting to the API

Communication with the API takes place via JavaScript files using the fetch function. All requests are sent in JSON format and point to the same base URL:

`http://localhost:8080/api/rest/pingu/`

Each screen is responsible for making its own calls to the API depending on the functionality it implements. For example, login is performed via a POST request to the `/auth/login` endpoint.

Example of a call:

```
fetch("http://localhost:8080/api/rest/pingu/auth/login", {  
  method: "POST",  
  headers: { "Content-Type": "application/json" },  
  body: JSON.stringify({  
    alias: usuario,  
    contrasena: password  
  })  
})
```

When the server response is successful (status code 200), the user ID is stored in *localStorage*, which allows the session to remain active in the browser. From that point onwards, the user is redirected to the app's home screen. In the event of an error, a message is displayed stating that the credentials are incorrect.

## 5.4. Data validation

Before sending any request to the API, validation checks are carried out on the client side to prevent errors and improve the user experience.

If any field is incorrect or empty, the system displays a dynamic message within the input field itself using a placeholder, accompanied by a visual change to the field's border (red colour). These messages are selected at random from a predefined list, which adds a more dynamic touch to the interface.

Once the user types in the field again, the style is automatically reset, allowing them to continue entering data.

## 6. Mobile App

The PingU mobile app provides access to all the main features from Android devices. It is a native app that replicates the behavior of the web version, adapted for touch interaction and the mobile environment.

### 6.1. Technology used

The application has been developed as a native Android app in Java, using the Android Studio development environment. The Retrofit library has been used to communicate with the API, which simplifies the management of HTTP requests and the processing of responses in JSON format.

The use of Retrofit allows endpoints to be clearly defined and network logic to be separated from the rest of the application, making the code easier to maintain.

### 6.2. Project structure

The application is organized into different activities, each representing a specific screen within the application. This structure follows a modular approach, where each functionality is separated into its own class.

| Class                            | Function                       |
|----------------------------------|--------------------------------|
| MainActivity                     | Login screen                   |
| MainActivity1Home                | Feed with posts from all users |
| MainActivity2perfil              | View and edit user profile     |
| MainActivity3recuperarContrasena | Recover password               |
| MainActivity4registro            | Register for a new account     |
| MainActivity5muro                | User´s personal wall           |
| MainActivity6riddlerCompany      | Company screen                 |
| MainActivity7followers           | List of followers              |
| MainActivity8follows             | List of people you follow      |

|                              |                          |
|------------------------------|--------------------------|
| MainActivity9formulario_post | Form for creating a post |
| MainActivity10Usuarios       | List of all users        |

List of all users. In addition to the activities, there are helper classes that handle communication with the API and the display of data on screen:

- **ApiClient:** configures Retrofit with the API's base URL
- **ApiService:** defines all available endpoints
- **PostAdapter / UsersAdapter:** adapters used in RecyclerView to display dynamic lists

This structure allows for a clear separation of interface logic, network logic and data management.

### 6.3. Connecting to the API

The connection to the API is made using Retrofit, configured in the *ApiClient* class. The base URL used is:

`http://10.0.2.2:8080/api/rest/pingu/`

This address is used in the Android emulator, as 10.0.2.2 redirects to the computer's *localhost*. To carry out tests on the device, the base URL used was: `http://192.168.1.29:8080/api/rest/pingu/`

The endpoints are defined in the *ApiService* interface, where each method represents a call to the API. Retrofit automatically converts the JSON responses into Java objects, which simplifies data handling within the application.

### 6.4. Session Management

The logged-in user's ID is stored locally using *SharedPreferences*, with the key `id_usuario`. This allows the session to remain active even as the user navigates between different screens.

Example of user retrieval:

```
getSharedPreferences('PinguPrefs', MODE_PRIVATE).getString("id_usuario", "");
```

Thanks to this mechanism, all activities can easily access the current user without the need for continuous re-authentication.

### 6.5. Validations

As with the web application, the data entered by the user is validated before any API call is made. This prevents unnecessary errors and improves the user experience.

When a field is incorrect or empty, an error message is displayed, selected at random from a predefined list. This approach makes the interaction more dynamic and less repetitive for the user.

Furthermore, validations are applied on various screens, such as login, registration or post creation, ensuring that the data sent to the API is correct.



## 7. Desktop application

The PingU desktop application is designed to run on Windows systems and offers a comprehensive social network experience via a form-based graphical interface. This client allows users to interact with all the system's features in a similar way to the web and mobile versions.

### 7.1. Technology used

The application has been developed in C# using Windows Forms on the .NET Framework 4.7.2. This approach allows graphical interfaces to be created quickly using forms and visual controls.

The HttpClient class is used to communicate with the API, making requests asynchronously via *async/await*. This is particularly important in desktop applications, as it prevents the interface from freezing whilst waiting for a response from the server.

### 7.2. Project structure

| Form               | Function                            |
|--------------------|-------------------------------------|
| Log_in             | Login screen                        |
| Registro           | Create a new account                |
| Principal          | Feed featuring posts from all users |
| Perfil             | View and edit profile details       |
| MisPost            | View the list of my posts           |
| NuevoPost          | Form from create a new post         |
| Recuperar_password | Reset password via email            |
| Registro           | Create a new user account           |
| Riddler            | Company screen                      |
| Seguidores         | List of followers                   |
| Seguidos           | List of follows                     |

|          |                   |
|----------|-------------------|
| Usuarios | List of all users |
|----------|-------------------|

Each form manages its own logic and navigation, using `ShowDialog()` to open new windows and `Hide()` or `Close()` to control the flow between screens.

One of the most important parts of the application is the dynamic construction of the post feed. In the *Principal* form, posts are not defined statically, but are generated at runtime by creating Windows Forms controls (Panel, Label, Button, etc.). Each post is represented as a card that includes the content, the author, the number of likes, and actions such as liking, deleting or replying.

Furthermore, certain elements of the interface are dynamic. For example, the delete button only appears when the post belongs to the logged-in user, which introduces permission control logic directly into the interface.

### 7.3. Connecting to the API

The connection to the API is established from each form using `HttpClient` instances. The base URL used is:

`http://localhost:8080/api/rest/`

From this base, each form constructs the necessary routes to access the various API endpoints.

Requests are made asynchronously using *async* and *await*, which allows the application to remain responsive whilst waiting for the server's response. For example, loading posts or users does not block the interface, preventing the application from freezing.

Data received in JSON format is deserialised using the *System.Text.Json* library, converting it into C# objects that can be used directly within the application.

### 7.4. Session Management

The logged-in user's ID is stored as a static variable in the *Log\_in* class:

```
public static string idUsuarioLogado = "";
```

This approach allows any form to easily access the current user without the need to pass data between windows. This simplifies session management within the application.

### 7.5. Validations

Validations in the desktop application follow the same logic as in the other clients. Before sending any request to the API, the fields are checked to ensure they are correct.

The most important validations include:

- Required fields (username, password, etc.)
- Minimum password length (6 characters)
- Valid email format

In the event of an error, messages are displayed using *MessageBox.Show()*. As in the other versions, messages are selected at random from a list, making the experience more dynamic and less repetitive for the user.

## 8. Security

### 8.1. Security mechanisms

The application implements various basic security measures to protect both data and user operations.

Firstly, the authentication system is based on a username and password. When the user logs in, the API validates the encrypted credentials and returns the user ID, which is used across different clients to keep the session active. Advanced tokens such as JWT are not used, but session management is handled correctly on each client (localStorage, SharedPreferences or in-memory variables).

As for the database, the risk of SQL injection is mitigated by using prepared queries via the API, which prevents users from entering malicious code into input fields.

As for potential Cross-Site Scripting (XSS) attacks, as this is a controlled application with no rendering of dynamic HTML entered by the user, the risk is low. Nevertheless, data is treated as plain text on the client side.

Furthermore, passwords are not stored in plain text in the database; instead, they are encrypted before being saved, adding an extra layer of protection in the event of unauthorized access.

### 8.2. User and permissions management

The permissions system is simple but effective. Each user can only modify or delete their own information and their own posts.

For example:

- Only the author of a post can delete it.
- Each user can only edit or delete their own profile.
- Actions such as liking or following other users are linked to the logged-in user's ID.

There are no roles in the system, as the application is designed as a basic social network where all users have the same permissions.

### 8.3. Best practices

During development, several best practices were followed to improve the security and stability of the application:

- Data validation on both the client and server before processing any request.
- Use of passwords with a minimum length to prevent weak credentials.
- Clear separation between client and server, preventing direct access to the database from clients.
- Controlled error handling, preventing sensitive information from being displayed to the user.
- Use of asynchronous requests to prevent deadlocks and unexpected behavior.

These measures ensure that the application operates securely within its intended scope.

## **9. Deployment**

### **9.1. How to run each application**

Each of the system's clients is run from its corresponding development environment.

The web application can be opened directly in the browser by accessing the web project's main file. For it to work correctly, the API must already be running locally, as all screens make requests to its endpoints.

The mobile application is run from Android Studio, using an Android emulator or a connected physical device. In the case of the emulator, the connection to the API is made via the address 10.0.2.2, which points to the computer's localhost. If a physical device is used, this address must be replaced with the local IP of the machine on which the API is running.

The desktop application runs from Visual Studio using Windows Forms. During development, the application is launched directly from the environment, without the need to generate a separate executable. As with the other clients, it requires the API to be running in order to access the data.

### **9.2. Deploying the API**

The REST API runs on a local Apache Tomcat server, configured on port 8080. It is deployed in a local environment, so before using any of the clients, you must start Tomcat and check that the API is available.

The base URL used is:

`http://localhost:8080/api/rest/pingu/`

In this way, the API acts as a central access point for the three clients, managing both the system logic and the connection to the database.

### **9.3. Environments**

The project has been developed primarily in a local development environment, where both the API and the clients run on the same machine, except in the case of the Android emulator, which accesses the local server via a specific address. The only component deployed on the internet is the database, accessible via filess.io

## 10. Testing

### 10.1. Testing of the API

During development, manual tests were carried out on all API endpoints using the RESTer tool. These tests focused on verifying that each operation functioned correctly in isolation.

Both correct cases (happy path) and error situations have been validated, including:

- Logging in with correct and incorrect credentials
- User registration with valid data and with errors (empty fields, duplicates, etc.)
- Creating posts with and without content
- Deleting existing and non-existent posts
- Liking and unliking in different situations
- Following and unfollowing users
- Password recovery

Furthermore, it has been verified that the API returns appropriate status codes (200, 201, errors) and consistent messages in each case.

### 10.2. Client-side testing

Once the API had been validated, comprehensive tests were carried out across all three clients (web, mobile and desktop) to ensure the system functioned correctly end-to-end.

- Notable tests carried out include:
- Full login and logout flow
- Navigation between screens
- Creation, viewing and deletion of posts
- Interaction with other users (likes, follows, replies)
- Editing the user profile
- Password recovery

It has been verified that data synchronises correctly across the different clients, as they all use the same API.

### 10.3. Error testing and validation

We have tested not only the correct scenarios, but also situations in which the application must fail in a controlled manner. This has enabled us to identify errors and improve the system's robustness.

Some examples:

- Empty required fields
- Passwords that are too short
- Incorrectly formatted data
- Attempts to perform actions without an active session
- API responses with errors

On the client side, these errors are handled by displaying clear messages to the user (many of them using randomised phrases to improve the user experience).

## 11. User and Maintenance Guide

### 11.1. Local execution

To run the project correctly locally, you must first start the API and then any of the clients.

1. Starting the API

- Start the Apache Tomcat server on port 8080
- Check that the API is responding at:
- <http://localhost:8080/api/rest/pingu/>

2. Checking the database

- The database has been created online at [filess.io](https://filess.io/). It is recommended that you check that it is working correctly and remains up and running.

3. Running the clients

- Web: open the index.html file in your browser
- Android: run from Android Studio (emulator or physical device)
- Desktop: run the project from Visual Studio

It is important that the API and the database are always running before starting any client, as they all depend on them.

### 11.2. Adding new features

The system is designed in a modular way, which makes it easy to add new features.

To add a new feature, the standard process is as follows:

1. Backend (API)

- Create the new endpoint in the API
- Add the necessary logic and, if required, modify the database

2. Client

- Call the new endpoint from the relevant client
- Add the necessary interface (screens, buttons, forms, etc.)

As the client and server are separate, it is possible to modify or extend one part without directly affecting the others, provided the API contract is maintained. In fact, it is always recommended to make changes to the contract first, so that subsequently both parties (API and clients) simply comply with them by performing the appropriate checks and calls.

### 11.3. Common issues

During the use and development of the project, some common issues may arise:

- The application does not load data: The API is not running or the base URL is incorrect
- Connection error on Android: Incorrect use of localhost instead of 10.0.2.2 in the emulator
- Login errors: Incorrect credentials or unregistered user
- Incorrect date data (long numbers): Formatting issue (timestamp in milliseconds instead of a readable date)
- Failure when modifying or deleting data: The user lacks permissions (for example, attempting to delete a post that does not belong to them)
- Database issues: Tables not created or foreign keys configured incorrectly

## **11.4. Maintenance**

System maintenance mainly consists of:

- Checking that the API continues to function correctly
- Verifying the integrity of the database
- Adapting clients if changes are made to the endpoints
- Correcting errors detected during testing or in use

Thanks to the client-server architecture, any improvements to the API are automatically reflected across all clients, which facilitates the overall maintenance of the system.



## 12. Future improvements

Although the application is fully functional, there are several areas for improvement and expansion that would enable it to evolve both functionally and technically.

One of the first improvements would be to fully standardise the features across the three client versions. At present, some features may be implemented in some versions but not in others.

Another important improvement is image management. The database already includes fields for storing image URLs, and a preliminary version of this functionality exists in the mobile app. The next step would be to integrate a comprehensive system that allows:

- Uploading and editing the user's profile photo
- Adding images to posts

To achieve this, it would be necessary to use an image storage service, rather than saving files directly to the database.

A badge or reward system (already proposed in the original December document) could also be incorporated to encourage user participation.

Another interesting feature would be to allow users to see who has liked a post, adding more social interaction and transparency within the platform.

At the interface level, the plan is to continue improving the design and user experience across the three clients, aiming for interfaces that are more intuitive, modern and consistent with one another.

Finally, on a technical level, the API could be deployed on a remote server accessible from any network.